RAG: Retrieval-Augmented Generation

Why Retrieval?

“What is the class temperature now?”

AI without sensor data

“I need a sensor reading to answer.”

AI with sensor data

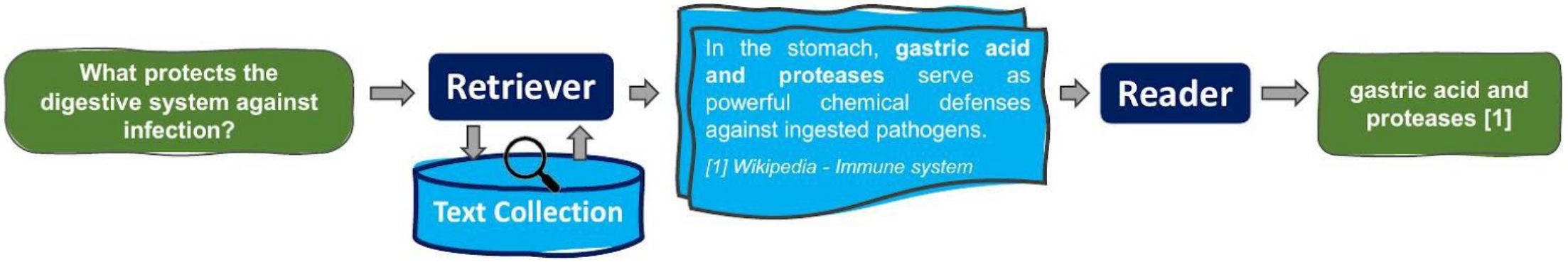
Sensor: 29 °C at 10:00 AM.

“The class sensor reads 29 °C.”

Retrieval means getting the information needed to answer.

Using Retrieval to Overcome LMs' Shortcomings

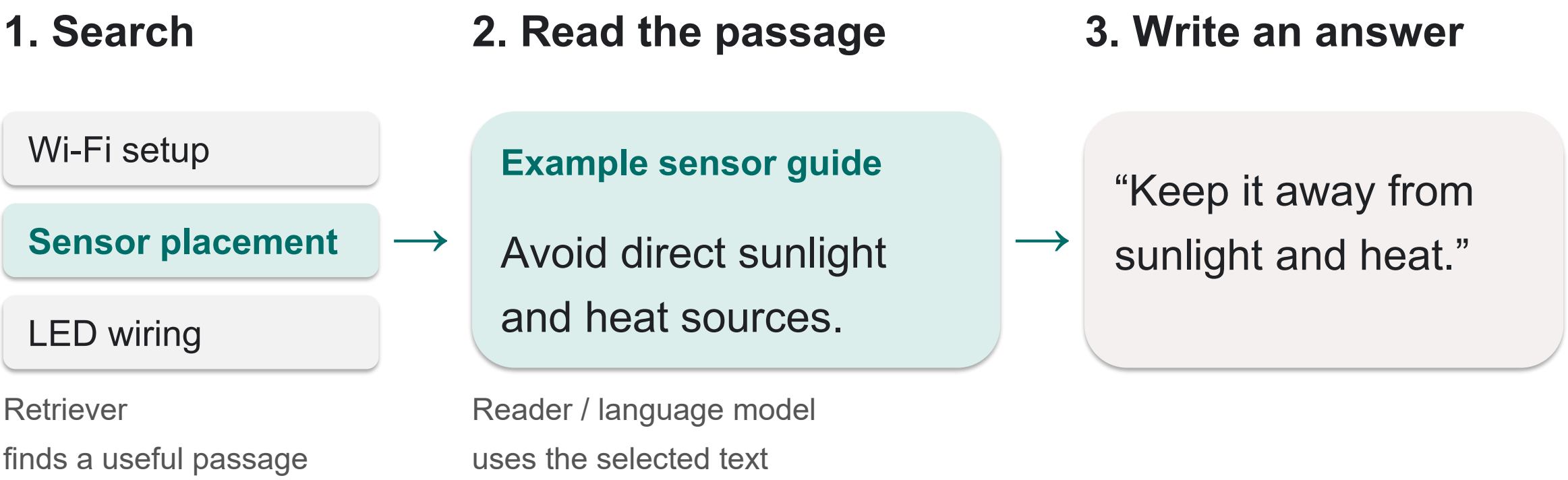
- Instead of asking the LM to memorize everything, can we provide the LM with relevant and useful content just-in-time?
- Retrieval is a common mechanism for identifying such relevant information.
  - **Dynamic**: it's easy to update / add documents to your retrieval system
  - **Interpretable**: LM can generate pointers to retrieved documents that support human verification of its generations (citations)



Answering IoT questions with documents

The assistant searches IoT guides before answering.

Question: “Where should we put our temperature sensor?”



The assistant finds a useful instruction, then answers in simple words.

Lecture Overview

- Why RAG (5 mins)
- The RAG pipeline (10 mins)
- Indexing: chunking, contextual retrieval (15 mins)
- Failure modes (15 mins)
- Going further: vector stores, multimodal, evaluation (15 mins)

RAG has two main jobs

Question: “Where should we put our temperature sensor?”

Retriever = finder

Searches the guides.

Finds a useful passage.

→

Generator = answer writer

Reads the question and passage.

Writes a clear answer.

One finds the information. The other explains it.

One IoT question, from search to answer

“Where should we put our temperature sensor?”

1

Retrieve

Find the sensor placement guide.

→

2

Add evidence

Give the model the question + guide text.

→

3

Generate

Write an answer using that text.

RAG = retrieve information, add it to the input, then generate an answer.

Retriever: find the useful passage

Question: “Where should we put our temperature sensor?”

Available guides

Wi-Fi setup

Sensor placement

LED wiring

→

Selected guide passage

“Avoid direct sunlight and heat sources.”

Return the useful instruction, not every page.

Generator: turn the passage into an answer

What the model receives

Question

Where should we put the temperature sensor?

Guide passage

Avoid direct sunlight and heat sources.

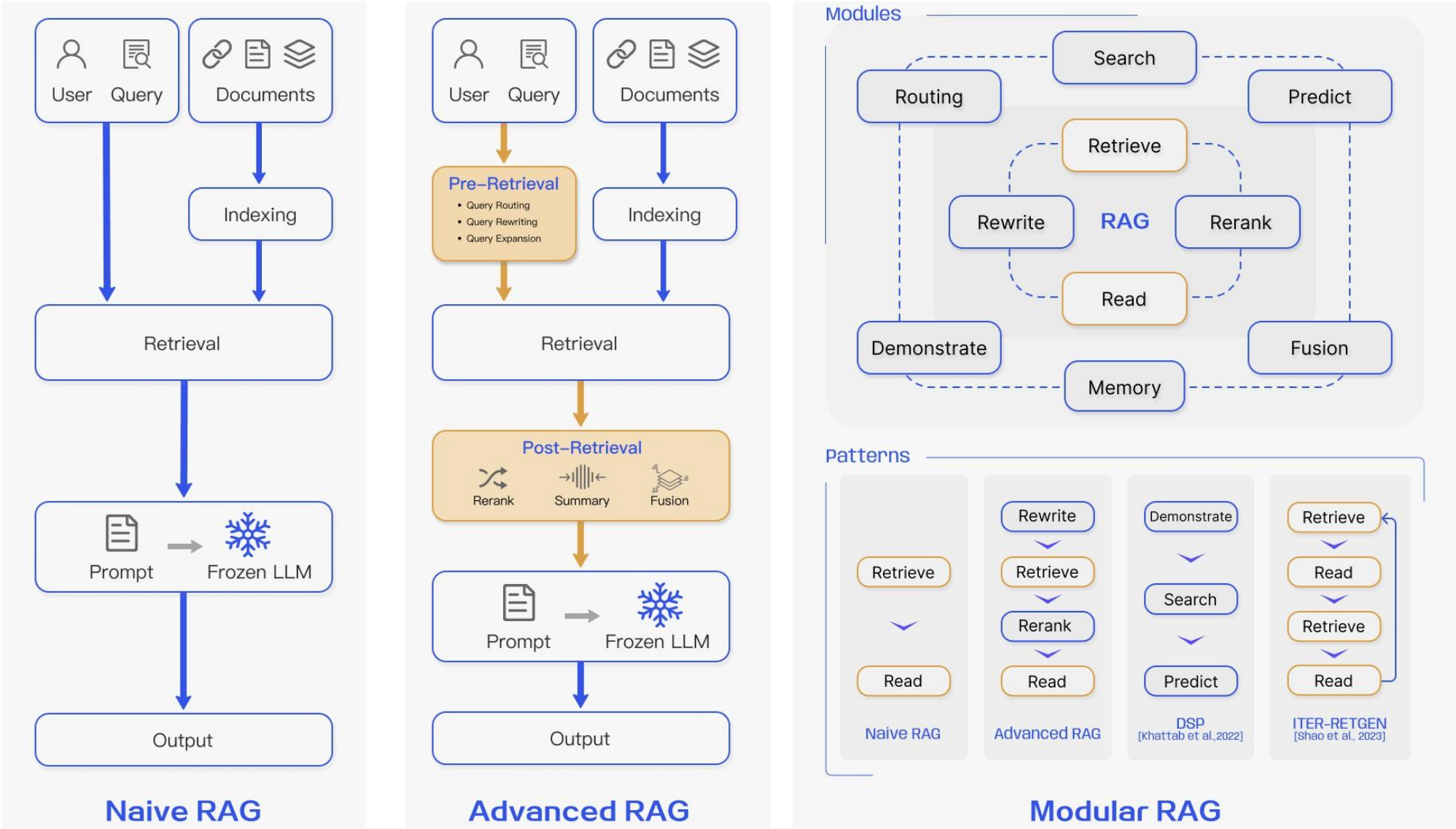
→

The answer

“Keep the sensor away from sunlight and heat.”

Use the evidence. If it is missing, say so.

Different Variants of RAG (Gao et al, 2024)



Lecture Overview

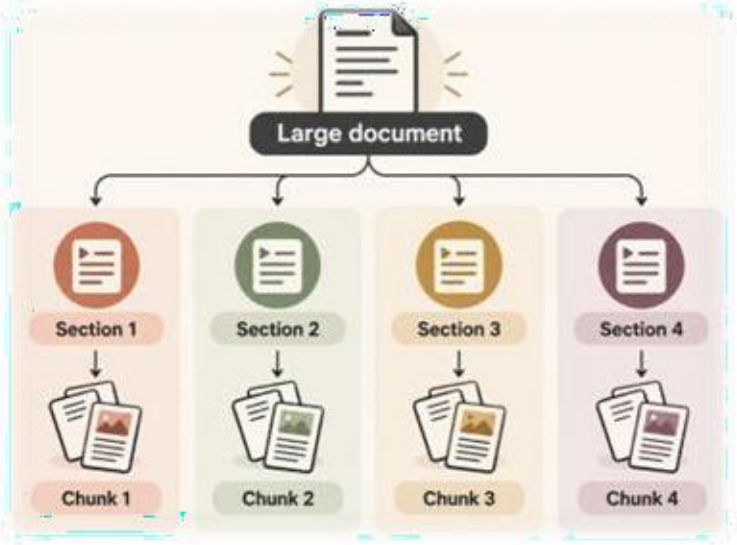
- Why RAG (5 mins)
- The RAG pipeline (10 mins)
- Indexing: chunking, contextual retrieval (15 mins)
- Failure modes (15 mins)
- Going further: vector stores, multimodal, evaluation (15 mins)

What Makes A Good Retriever? Embeddings

Type	Examples	One vector covers
Static word	word2vec, GloVe	a word type
Contextual	BERT, RoBERTa	a token in context
Passage	Sentence-BERT, DPR, E5	a chunk
Multimodal	CLIP	text or image, shared space
Graph	node2vec, GNN encoders	a node or relation

Retrieval needs both a fixed-dimension vector per chunk, and a similarity metric (cosine or inner product).

Chunking: Basic Unit in Retrieval



Fixed-size chunking:

- Segment based on size (e.g., 100–500 tokens)
  - *Too small:* the vector has no context
  - *Too large:* one vector averages many topics
- *Overlap:* 10–20%; no sentence lost at chunk boundary

Semantic chunking: segment via logical boundaries (e.g., sentences, sections)

Context-enriched chunking: carry additional metadata or summaries per chunk

AI-driven dynamic chunking: use LLMs to detect natural breakpoints

https://community.databricks.com/t5/technical-blog/the-ultimate-guide-to-chunking-strategies-for-rag-applications/ba-p/113089

Chunking: Same Text, Three Splits

```
Grading. The project milestone is due Week 6, Thursday 11:59pm. Late days may not be applied to the milestone. The final report is due Week 10.
```

Fixed-size chunking (50 tokens)

```
… The project milestone is due Week 6, Thursday
```

```
11:59pm. Late days may not be applied to the milestone …
```

Fixed-size chunking + overlap

```
… The project milestone is due Week 6, Thursday 11:59pm.
```

```
… due Week 6, Thursday 11:59pm. Late days may not be applied …
```

Structure (split on headings)

```
Grading — Milestone: Week 6, Thu 11:59pm. Late days: not allowed on the milestone. Final report: Week 10.
```

Why Contextual Retrieval

Query: “What did our lab sensor read at 10:00 AM?”

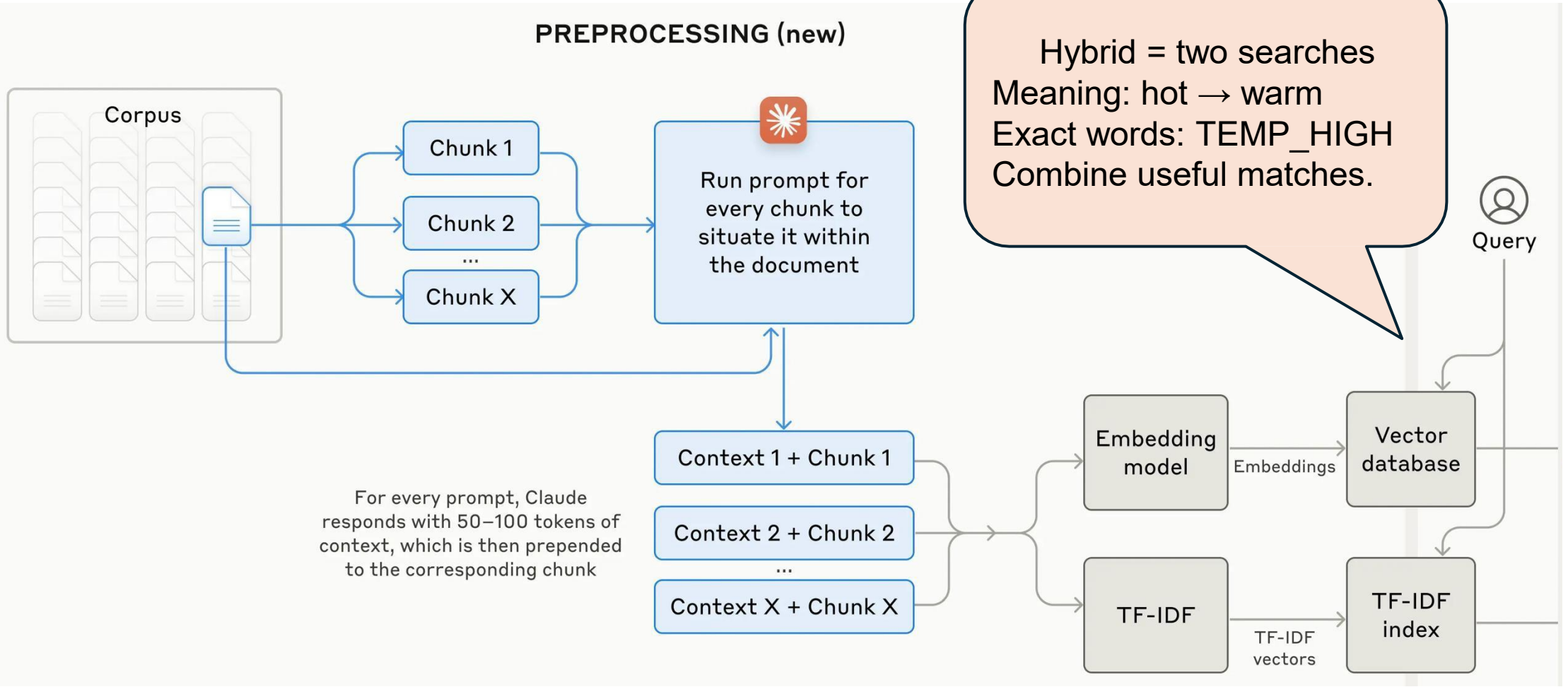
Chunk

“The temperature was 29 °C.”

Contextualized Chunk

“Lab Sensor A, IoT Lab.
Recorded at 10:00 AM.
The temperature was 29 °C.”

Contextual Retrieval



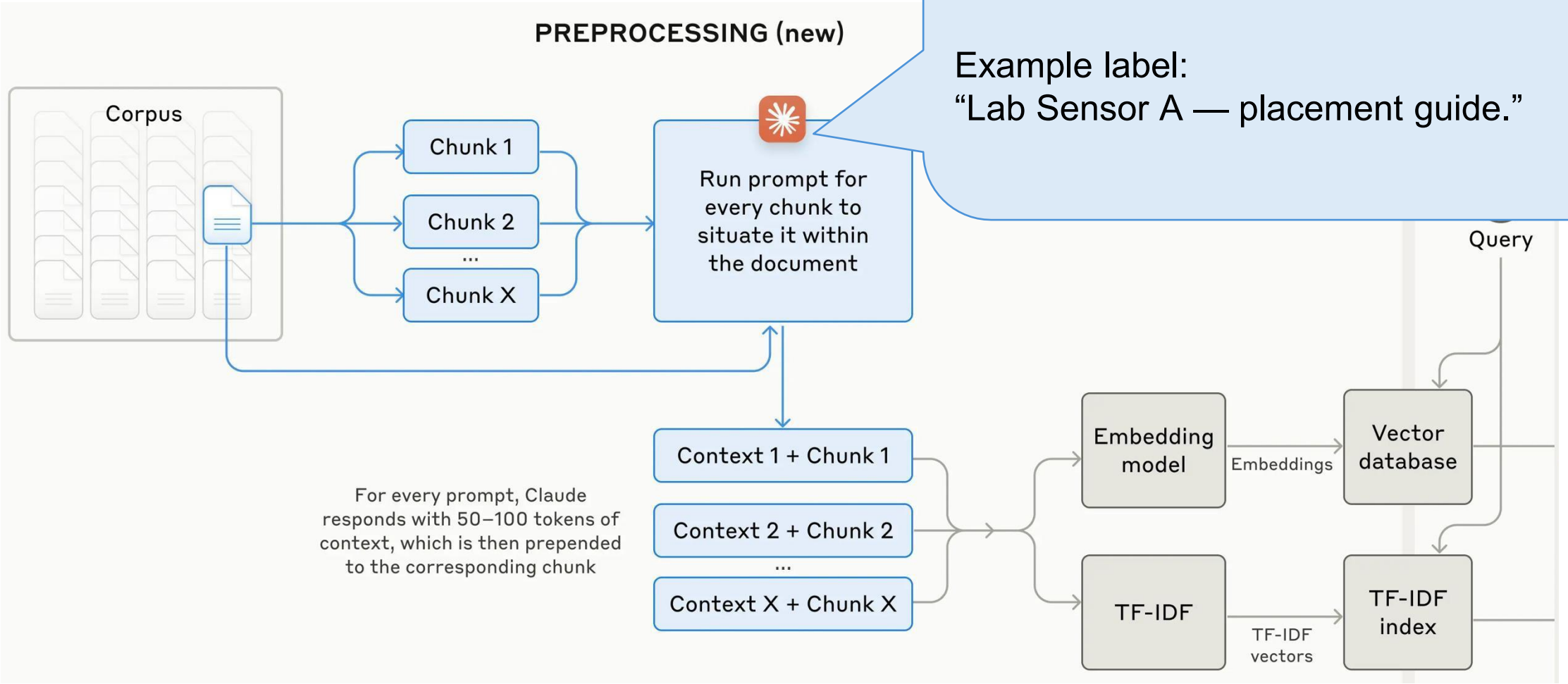
https://www.anthropic.com/engineering/contextual-retrieval

Contextual Retrieval

What we ask the model to do

Read the guide and this passage.
Add a short label: which sensor and topic?

Example label:
“Lab Sensor A — placement guide.”

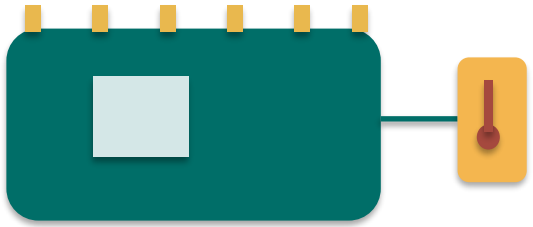


https://www.anthropic.com/engineering/contextual-retrieval

Vector search: find guides with similar meaning

A vector is a list of numbers used to represent meaning.
Vector search compares the question with stored guide passages.

“Why is my sensor hot?”



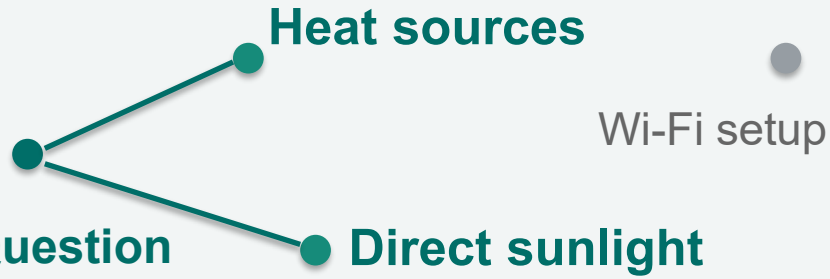
IoT sensor node

→

Turn the question into numbers

→

Guide passages: a map of meaning



Similar meaning → useful passages.

Lecture Overview

- Why RAG (5 mins)
- The RAG pipeline (10 mins)
- Indexing: chunking, contextual retrieval (15 mins)
- Failure modes (15 mins)
- Going further: vector stores, multimodal, evaluation (15 mins)

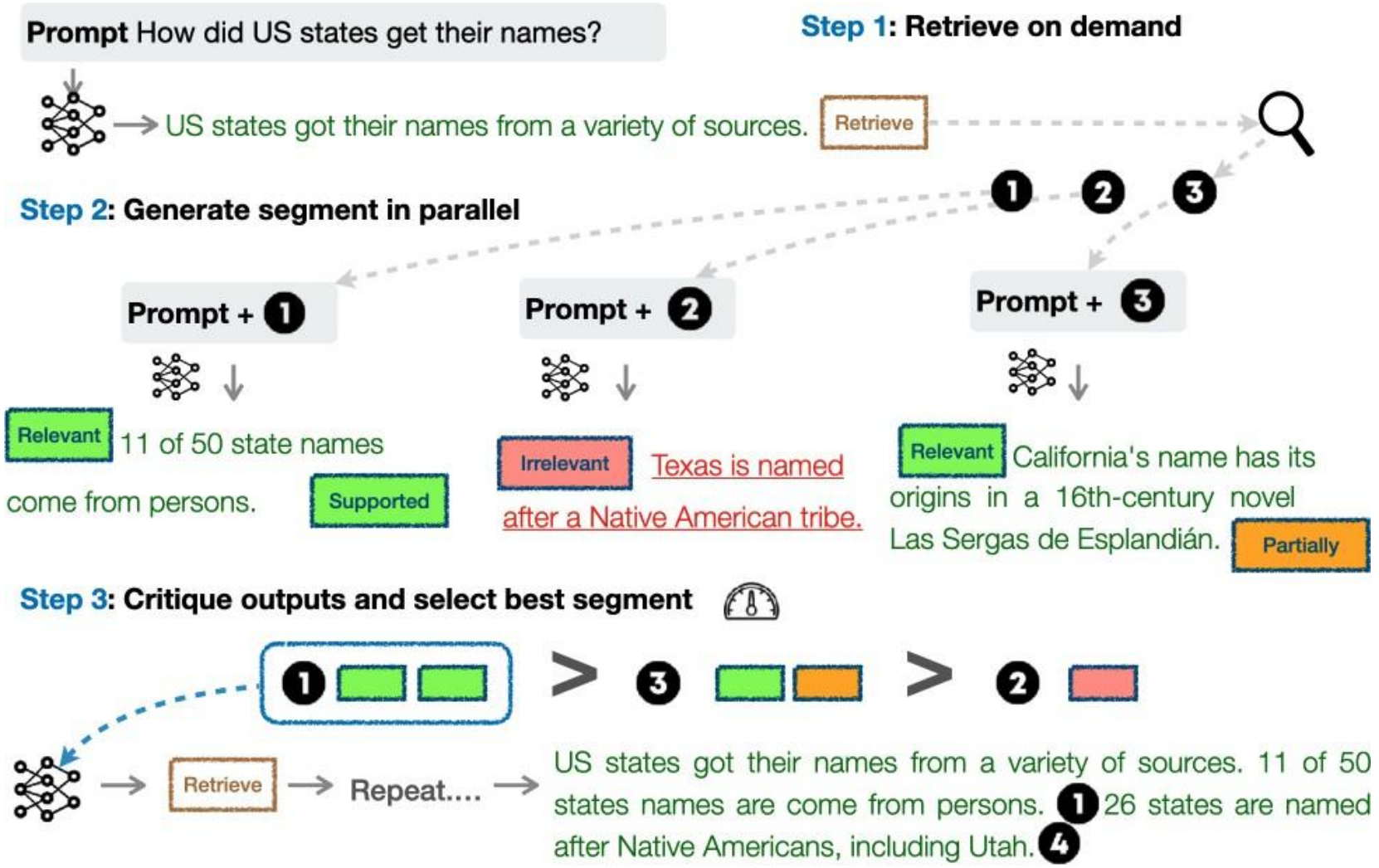
Agentic RAG: Retrieval inside the Loop

Everything so far answers one question well. None of it decides **whether** to retrieve, **what** to ask, or **when to stop**.



- **Retrieval should be an action the agent takes**
- **Three decisions:** (1) retrieve or answer directly; (2) what query to issue; (3) whether the result is good enough

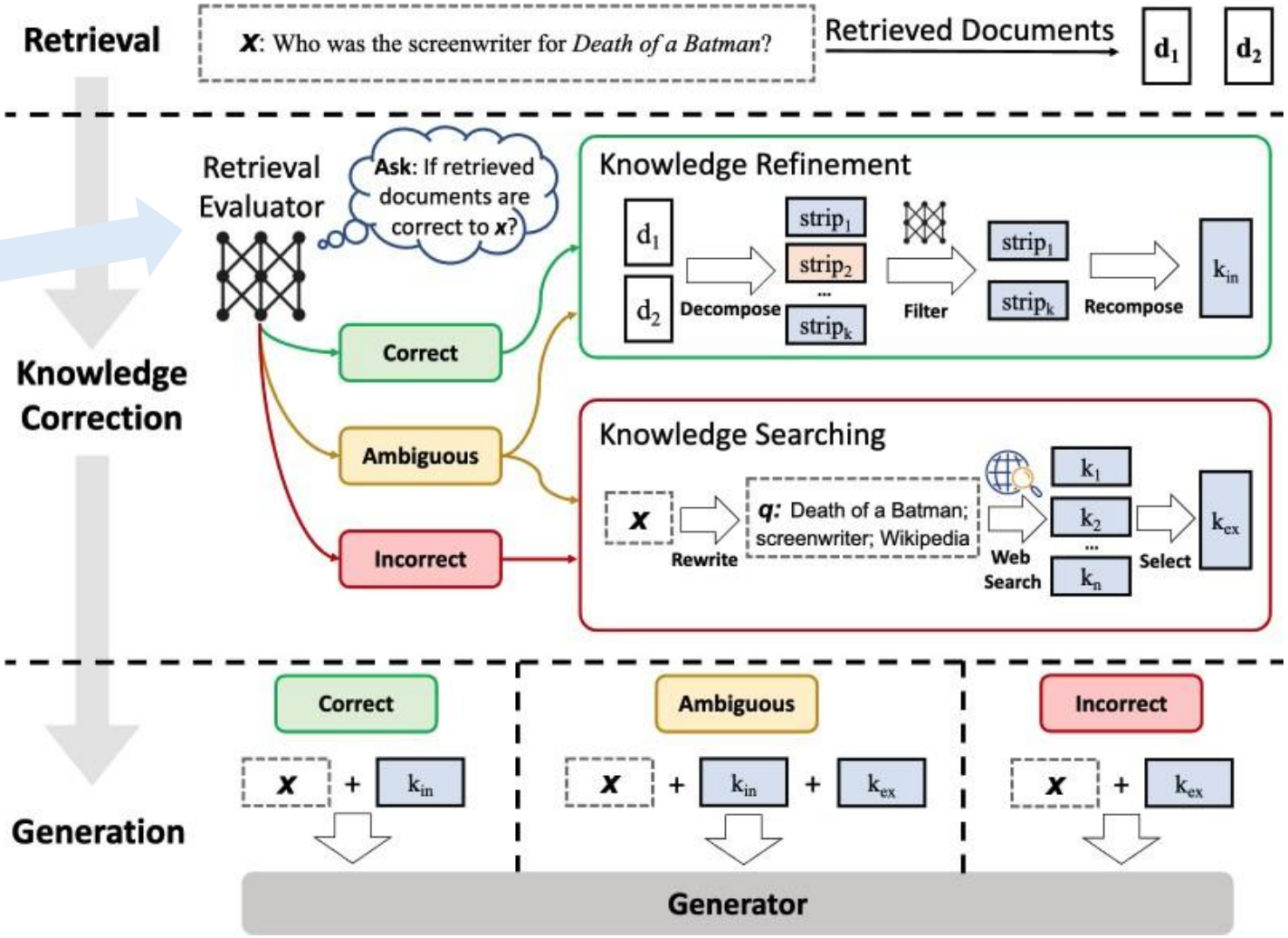
Self-RAG: Self-reflective RAG (Asai et al., 2024)



SELF-RAG learns to retrieve, critique, and generate text passages to enhance overall generation quality, factuality, and verifiability.

CRAG: a critic grades the retrieval (Yan et al., 2024)

To evaluate the relevance of the retrieved docs to the input, and estimate a confidence based on which different knowledge retrieval actions can be triggered.



Lecture Overview

- Why RAG (5 mins)
- The RAG pipeline (10 mins)
- Indexing: chunking, contextual retrieval (15 mins)
- Failure modes (15 mins)
- Going further: vector stores, multimodal, evaluation (15 mins)

Issues with RAG

Retrieval-side



Recall ceiling
Missing evidence limits agent

Stale index
Embeddings not update

Permission leak
Use content not allowed to access

Reader-side

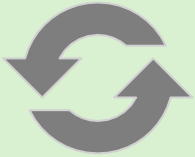


Lost in the middle
Position bias

Distraction
Similar but irrelevant content

Evidence conflict
Sources or knowledge conflicts

Loop-side



Compounding error
Early mistakes affect later decision

No stopping rule
Lack clear conditions for finishing

Prompt injection
Retrieved text steers the agent

Lecture Overview

- Why RAG (5 mins)
- The RAG pipeline (10 mins)
- Indexing: chunking, contextual retrieval (15 mins)
- Failure modes (15 mins)
- Going further: vector stores, multimodal, evaluation (15 mins)

Vector Databases

Specialised storage for high-dimensional embeddings

The Problem

- Standard databases compare rows by value — not by meaning.
- Searching 'sensor overheating' won't match 'device running hot'.
- Need a store that compares geometry, not strings.

→

The Solution

- Embed text as vectors (lists of numbers).
- Store vectors in an index built for nearest-neighbour search.
- At query time: embed the question → find the closest vectors.
- Returns semantically similar passages, not keyword matches.

Embeddings convert language into geometry — similar meaning ≈ nearby point.

Choosing a Vector Store

Five common options — each suits a different stage of production.

System	Hosting	Best For
Pinecone	Managed SaaS	Production, serverless scale
Weaviate	Self-host / Cloud	Hybrid search + GraphQL API
ChromaDB	Local / embedded	Prototyping, dev laptops
Qdrant	Self-host / Cloud	High-throughput, rich filtering
pgvector	Postgres ext.	Existing Postgres infra

Most of these use an HNSW index (Hierarchical Navigable Small World): fast approximate search, high memory.

Multimodal RAG

Real documents contain more than text

Text

Paragraphs, headings, bullet lists.
Standard embedding pipeline.
Works well with BM25 + dense.

Images & Figures

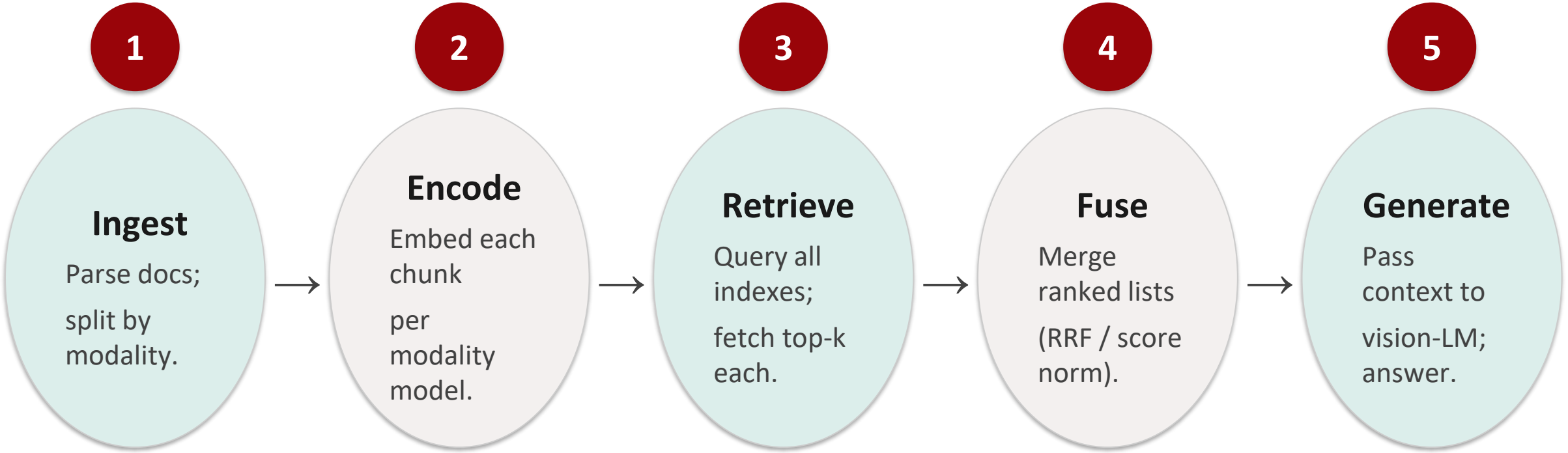
Diagrams, photos, charts in documents.
CLIP or ColPali for embedding.
No OCR needed with ColPali.

Tables & PDFs

Structured data, scanned pages.
Layout-aware parsers (e.g. Unstructured.io).
Embed rows or page images.

ColPali (2024) — embed full PDF pages as images; skip OCR entirely.

Multimodal RAG — Pipeline



Each modality uses its own encoder; results fuse before the LLM sees them.

RRF = Reciprocal Rank Fusion — a simple, effective rank-merging formula.

Evaluating RAG is Harder Than It Looks

End-to-end accuracy hides failures

A correct answer can hide bad retrieval (lucky generation).

A wrong answer can come from perfect retrieval + bad generation.

You need separate signals for each stage.

Retrieval quality ≠ answer quality

Did we fetch the right chunks? → Context Precision / Recall

Did the model use them faithfully? → Faithfulness

Did the answer address the question? → Answer Relevance

Ground truth is expensive

Human annotation of relevant passages does not scale.

LLM-as-judge (e.g. RAGAS) offers automatic, reference-free eval.

Bias in the judge model can still leak into scores.

Evaluate retrieval and generation independently — end-to-end accuracy alone is misleading.

RAGAS: Automated RAG Evaluation

Es et al., 2023 • pip install ragas

Context Precision

Are the retrieved chunks actually relevant?

High precision = less noise in context.

Context Recall

Are all the needed facts covered?

High recall = model has enough evidence.

Faithfulness

Does every claim in the answer appear in the retrieved context?

Catches hallucination.

Answer Relevance

Does the answer actually address the question asked?

Catches off-topic responses.

Computed by an LLM judge. Faithfulness and Answer Relevance need no labels; Context Recall needs a reference answer.

RAG vs Fine-tuning

RAG

Knowledge lives outside the model.

Update the index; no retraining.

Answers can cite their sources.

Retrieval adds latency.

Quality depends on index + embeddings.

V
S

Fine-tuning

Knowledge baked into model weights.

Retraining needed when data changes.

No natural citation mechanism.

Faster inference — no retrieval step.

Better for style / format consistency.

RAFT (2024) combines both: fine-tune the model to reason over retrieved documents.

When to Use Each

Use RAG

- Knowledge changes frequently

(news, product docs, sensor logs).

- Answers must cite sources.
- No compute budget for retraining.
- Domain docs already exist.

Use Fine-tuning

- Consistent output style or format.
- Latency budget is tight.
- Knowledge is stable.
- No suitable doc corpus.

Use Both (RAFT)

- Complex reasoning over private,

frequently updated knowledge.

- High accuracy and auditability both required.
- You have compute + labelled examples.
- e.g. enterprise AI assistants.

When in doubt, start with RAG — faster to iterate, easier to debug.

Thank You